Sardar Vallabhbhai National Institute of Technology (SVNIT),
Surat Department of Artificial Intelligence
Agentic AI (IA462) , Semester VIII
U23AI052 Submission

Experiment 1

Simple Reflex (Reactive) Agent — The Vacuum-Cleaner World

Aim

To implement a reactive agent that maps percepts directly to actions through condition–action rules, and to measure its behaviour with a performance score in a simulated environment.

Theory

An agent is anything that perceives its environment through sensors and acts upon it through actuators. The simplest architecture is the simple reflex agent: a fixed set of condition–action rules maps the current percept straight to an action, with no internal state, no model of the world and no look-ahead. Russell and Norvig's vacuum-cleaner world is the canonical testbed: two locations that may be dirty, an agent that can move and suck, and a performance measure that rewards cleaning and penalises movement.

The experiment also establishes the agent–environment separation that every later experiment reuses: the environment owns the true state and exposes only percept() and execute(action); the agent sees nothing but the percept. Note the limitation visible in the sample output — once both squares are clean the reflex agent oscillates forever, bleeding performance, because it cannot remember that the other square was already clean. Fixing this requires internal state (Experiment 2).

Table 1. PEAS description of the task environment.

PEAS component	Specification for this experiment
Performance measure	+1 for each clean square at each time step; −1 for every movement (energy cost)
Environment	Two squares A and B, each Clean or Dirty; deterministic, fully observable locally
Actuators	Suck, Left, Right, NoOp
Sensors	Location sensor and dirt sensor, giving the percept (location, status)

Algorithm

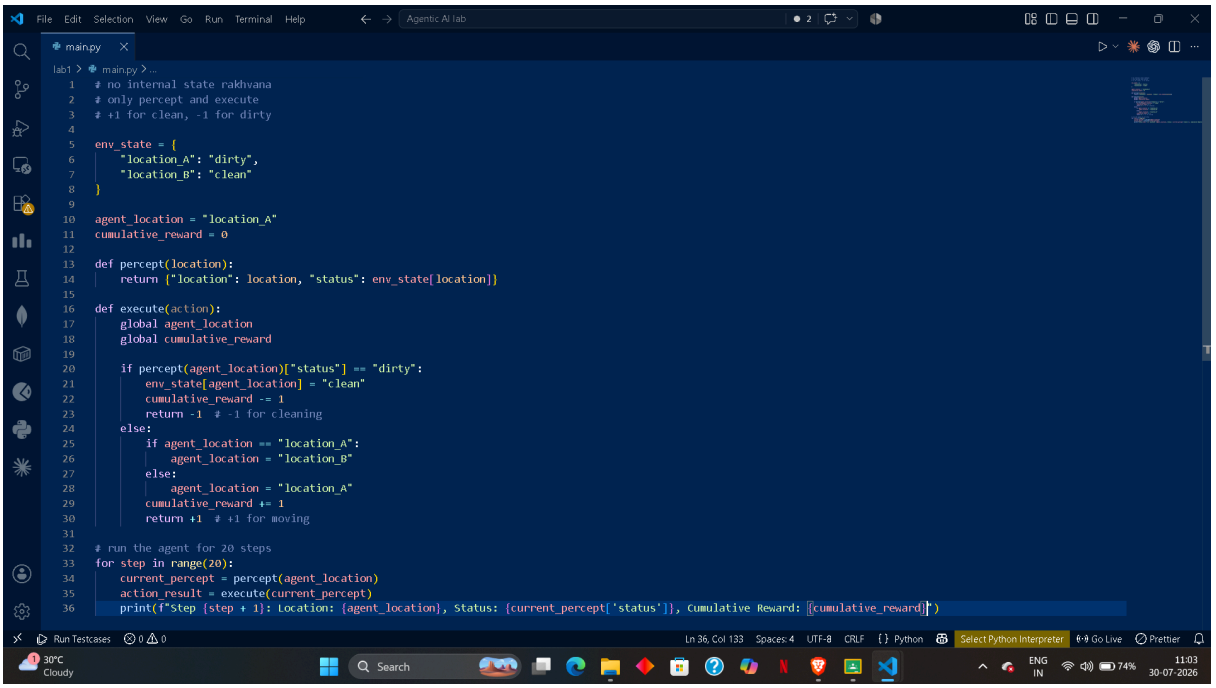
1. Build an environment class holding the dirt status of both locations, the agent's location

and a performance score.

2. Implement percept() returning (location, status) and execute(action) applying suck/left/right with rewards +10 and −1.

3. Implement the agent as a pure function: if dirty then suck, else move to the other square.

4. Run the perception–action loop for a fixed number of steps and report the final state and score.



Procedure and Result

Paste the snapshots of the steps followed and the output produced by the simple reflex agent below, including the initial state, the percept–action trace of every step, and the final state with the performance score.

```
D:\Agentic AI lab>python lab1/main.py
Step 1: Location: location_A, Status: dirty, Cumulative Reward: -1
Step 2: Location: location_B, Status: clean, Cumulative Reward: 0
Step 3: Location: location_A, Status: clean, Cumulative Reward: 1
Step 4: Location: location_B, Status: clean, Cumulative Reward: 2
Step 5: Location: location_A, Status: clean, Cumulative Reward: 3
Step 6: Location: location_B, Status: clean, Cumulative Reward: 4
Step 7: Location: location_A, Status: clean, Cumulative Reward: 5
Step 8: Location: location_B, Status: clean, Cumulative Reward: 6
Step 9: Location: location_A, Status: clean, Cumulative Reward: 7
Step 10: Location: location_B, Status: clean, Cumulative Reward: 8
Step 11: Location: location_A, Status: clean, Cumulative Reward: 9
Step 12: Location: location_B, Status: clean, Cumulative Reward: 10
Step 13: Location: location_A, Status: clean, Cumulative Reward: 11
Step 14: Location: location_B, Status: clean, Cumulative Reward: 12
Step 15: Location: location_A, Status: clean, Cumulative Reward: 13
Step 16: Location: location_B, Status: clean, Cumulative Reward: 14
Step 17: Location: location_A, Status: clean, Cumulative Reward: 15
Step 18: Location: location_B, Status: clean, Cumulative Reward: 16
Step 19: Location: location_A, Status: clean, Cumulative Reward: 17
Step 20: Location: location_B, Status: clean, Cumulative Reward: 18
```

Observations

Comment on why the agent keeps moving after both squares are clean, on how the performance score changes with the number of steps, and on what the result tells you about the need for internal state.

What i did is exactly as written, +1 was reward for moving, as the state is already clean, -1 for staying, and cumulative reward i added extra just for seeing how the reward progression works

In overall view the agent is reactive and hence reward doesn't matter at all, only state's status do i.e dirty or clean

Exercises

1. Add a model-based variant that remembers the status of both squares and executes a no-op once everything is known to be clean; compare scores over 20 steps.

```python
def percept(location):
    return {"location": location, "status": env_state[location]}

def execute(per):
    global agent_location
    global cumulative_reward

    location = per["location"]
    status = per["status"]

    # update memory
    model[location] = status

    # if everything known clean -> NoOp
    if model["location_A"] == "clean" and model["location_B"] == "clean":
        cumulative_reward += 1
        return "NoOp"

    if status == "dirty":
        env_state[location] = "clean"
        model[location] = "clean"
        cumulative_reward -= 1
        return "Suck"

    if agent_location == "location_A":
        agent_location = "location_B"
    else:
        agent_location = "location_A"

    cumulative_reward += 1
    return "Move"

for step in range(20):
    p = percept(agent_location)
    action = execute(p)

    print(f"Step {step+1}: Action={action}, Location={agent_location}, Reward={cumulative_reward}")
```

```
D:\Agentic AI lab>python lab1/model.py
Step 1: Action=Suck, Location=location_A, Reward=-1
Step 2: Action=Move, Location=location_B, Reward=0
Step 3: Action=NoOp, Location=location_B, Reward=1
Step 4: Action=NoOp, Location=location_B, Reward=2
Step 5: Action=NoOp, Location=location_B, Reward=3
Step 6: Action=NoOp, Location=location_B, Reward=4
Step 7: Action=NoOp, Location=location_B, Reward=5
Step 8: Action=NoOp, Location=location_B, Reward=6
Step 9: Action=NoOp, Location=location_B, Reward=7
Step 10: Action=NoOp, Location=location_B, Reward=8
Step 11: Action=NoOp, Location=location_B, Reward=9
Step 12: Action=NoOp, Location=location_B, Reward=10
Step 13: Action=NoOp, Location=location_B, Reward=11
Step 14: Action=NoOp, Location=location_B, Reward=12
Step 15: Action=NoOp, Location=location_B, Reward=13
Step 16: Action=NoOp, Location=location_B, Reward=14
Step 17: Action=NoOp, Location=location_B, Reward=15
Step 18: Action=NoOp, Location=location_B, Reward=16
Step 19: Action=NoOp, Location=location_B, Reward=17
Step 20: Action=NoOp, Location=location_B, Reward=18
```

We see no op once the state we have to move to is cleaned

2. Extend the world to a 1×N corridor of N squares with random initial dirt and adapt the rules.

```python
env_state = {}

for i in range(N):
    env_state[i] = random.choice(["clean", "dirty"])

def percept(location):
    return {"location": location, "status": env_state[location]}

def execute(per):
    global agent_location
    global direction
    global cumulative_reward

    if per["status"] == "dirty":
        env_state[agent_location] = "clean"
        cumulative_reward -= 1
        return "Suck"

    next_location = agent_location + direction

    if next_location >= N:
        direction = -1
        next_location = N - 2

    elif next_location < 0:
        direction = 1
        next_location = 1

    agent_location = next_location
    cumulative_reward += 1
    return "Move"

for step in range(20):
    p = percept(agent_location)
    action = execute(p)

    print(f"Step {step+1}: Action={action}, Location={agent_location}, Reward={cumulative_reward}")
```

```
D:\Agentic AI lab>python lab1/corridor.py
Step 1: Action=Suck, Location=0, Reward=-1
Step 2: Action=Move, Location=1, Reward=0
Step 3: Action=Move, Location=2, Reward=1
Step 4: Action=Move, Location=3, Reward=2
Step 5: Action=Move, Location=4, Reward=3
Step 6: Action=Suck, Location=4, Reward=2
Step 7: Action=Move, Location=3, Reward=3
Step 8: Action=Move, Location=2, Reward=4
Step 9: Action=Move, Location=1, Reward=5
Step 10: Action=Move, Location=0, Reward=6
Step 11: Action=Move, Location=1, Reward=7
Step 12: Action=Move, Location=2, Reward=8
Step 13: Action=Move, Location=3, Reward=9
Step 14: Action=Move, Location=4, Reward=10
Step 15: Action=Move, Location=3, Reward=11
Step 16: Action=Move, Location=2, Reward=12
Step 17: Action=Move, Location=1, Reward=13
Step 18: Action=Move, Location=0, Reward=14
Step 19: Action=Move, Location=1, Reward=15
Step 20: Action=Move, Location=2, Reward=16
```

The agent simply walks left to right and back again while cleaning any dirty square it encounters.

3. Make dirt reappear with probability 0.1 per step and discuss whether the reflex agent is now rational.

```python
def percept(location):
    return {"location": location, "status": env_state[location]}

def add_new_dirt():
    for room in env_state:
        if env_state[room] == "clean":
            if random.random() < 0.1:
                env_state[room] = "dirty"

def execute(per):
    global agent_location
    global cumulative_reward

    if per["status"] == "dirty":
        env_state[agent_location] = "clean"
        cumulative_reward -= 1
        action = "Suck"
    else:
        if agent_location == "location_A":
            agent_location = "location_B"
        else:
            agent_location = "location_A"

        cumulative_reward += 1
        action = "Move"

    add_new_dirt()
    return action

for step in range(20):
    p = percept(agent_location)
    action = execute(p)

    print(f"Step {step+1}: Action={action}, Location={agent_location}, Reward={cumulative_reward}, State={env_state}")
```

```
D:\Agentic AI lab>python lab1/dirt_reappear.py
Step 1: Action=Suck, Location=location_A, Reward=-1, State={'location_A': 'clean', 'location_B': 'clean'}
Step 2: Action=Move, Location=location_B, Reward=0, State={'location_A': 'clean', 'location_B': 'clean'}
Step 3: Action=Move, Location=location_A, Reward=1, State={'location_A': 'clean', 'location_B': 'clean'}
Step 4: Action=Move, Location=location_B, Reward=2, State={'location_A': 'clean', 'location_B': 'dirty'}
Step 5: Action=Suck, Location=location_B, Reward=1, State={'location_A': 'dirty', 'location_B': 'clean'}
Step 6: Action=Move, Location=location_A, Reward=2, State={'location_A': 'dirty', 'location_B': 'clean'}
Step 7: Action=Suck, Location=location_A, Reward=1, State={'location_A': 'clean', 'location_B': 'clean'}
Step 8: Action=Move, Location=location_B, Reward=2, State={'location_A': 'clean', 'location_B': 'dirty'}
Step 9: Action=Suck, Location=location_B, Reward=1, State={'location_A': 'clean', 'location_B': 'clean'}
Step 10: Action=Move, Location=location_A, Reward=2, State={'location_A': 'clean', 'location_B': 'clean'}
Step 11: Action=Move, Location=location_B, Reward=3, State={'location_A': 'clean', 'location_B': 'clean'}
Step 12: Action=Move, Location=location_A, Reward=4, State={'location_A': 'clean', 'location_B': 'clean'}
Step 13: Action=Move, Location=location_B, Reward=5, State={'location_A': 'clean', 'location_B': 'clean'}
Step 14: Action=Move, Location=location_A, Reward=6, State={'location_A': 'clean', 'location_B': 'clean'}
Step 15: Action=Move, Location=location_B, Reward=7, State={'location_A': 'clean', 'location_B': 'clean'}
Step 16: Action=Move, Location=location_A, Reward=8, State={'location_A': 'clean', 'location_B': 'clean'}
Step 17: Action=Move, Location=location_B, Reward=9, State={'location_A': 'clean', 'location_B': 'clean'}
Step 18: Action=Move, Location=location_A, Reward=10, State={'location_A': 'clean', 'location_B': 'clean'}
Step 19: Action=Move, Location=location_B, Reward=11, State={'location_A': 'clean', 'location_B': 'clean'}
Step 20: Action=Move, Location=location_A, Reward=12, State={'location_A': 'clean', 'location_B': 'clean'}
```

Yes, under this environment the reflex agent is still rational as dirt can appear at any time with probability 0.1. Therefore the environment is dynamic and a model-based agent that permanently performs NoOp after both rooms are clean would eventually leave newly appearing dirt uncleaned.